

Rapport de soutenance de fin d'alternance

Établissement : Epitech Bénin

Diplôme préparé : MSc Pro - spécialité *Cybersécurité / Cloud*

Promotion : Promo 2026 (2024-2026)

Titre : Conception et pilotage d'une plateforme d'intelligence de marché - CarPulse

Auteur : Martin DOHOU

Entreprise d'accueil : Bioénergie - Strasbourg, France

Fonction occupée : Ingénieur logiciel (alternance, en télétravail depuis Abomey-Calavi, Bénin)

Tuteur : Senad Dzudzevic

Remerciements

Je tiens à remercier l'ensemble des personnes qui ont contribué au bon déroulement de mon alternance et à la réussite du projet CarPulse.

Mes remerciements s'adressent en premier lieu à **Senad Dzudzevic**, mon tuteur, pour la confiance accordée en me confiant le pilotage de ce projet ainsi que pour son accompagnement tout au long de cette période.

Je remercie également l'ensemble de l'équipe pédagogique d'**Epitech Bénin** pour le suivi de mon alternance, ainsi que **Senad Dzudzevic** pour m'avoir accueilli au sein de l'entreprise.

Enfin, je remercie les membres de l'équipe projet - développeurs data, backend et frontend - qui m'ont fait remonter leurs avancements et avec qui la collaboration a été à la fois formatrice et stimulante.

Résumé

Ce rapport présente le projet **CarPulse**, une plateforme d'intelligence de marché automobile développée durant mon alternance au sein de l'entreprise **Bioénergie**. La plateforme collecte automatiquement plusieurs milliers d'annonces de véhicules, les analyse au travers de trois méthodes indépendantes de valorisation dont les résultats sont agrégés par un vote de consensus, et restitue les « bonnes affaires » via une application web sécurisée.

Au sein de ce projet, j'ai occupé une position transverse - **chef de projet, analyste fonctionnel et product manager** - tout en assurant personnellement la conception et le développement de la brique **backend et de la sécurité applicative**, ainsi que l'ensemble de la démarche **DevOps et Cloud** (conteneurisation, déploiement, isolation des services). Le fil conducteur de ce rapport, en cohérence avec ma formation - spécialité principale **cybersécurité** et spécialité secondaire **Cloud Computing** - est double : la **sécurisation de bout en bout** de la plateforme (authentification robuste, gestion des sessions et des appareils, cloisonnement des rôles, conformité) et son **industrialisation Cloud/DevOps** (architecture modulaire conteneurisée, réduction de la surface d'attaque, déploiement maîtrisé).

Mots-clés : cybersécurité, authentification, JWT, gestion de sessions, contrôle d'accès, RGPD, Cloud Computing, DevOps, conteneurisation, architecture logicielle, NestJS, pilotage de projet.

Abstract

This report presents CarPulse, an automotive market-intelligence platform developed during my work-study placement at the company Bioénergie. The platform automatically collects several thousand vehicle listings, analyses them through three independent valuation methods whose results are aggregated by a consensus vote, and surfaces the best deals through a secured web application.

Within this project, I held a cross-functional position - project manager, functional analyst and product manager - while personally designing and developing the backend and application-security component, as well as the entire DevOps and Cloud approach (containerisation, deployment, service isolation). In line with my degree - a major in cybersecurity and a minor in Cloud Computing - this report follows a twofold thread: the end-to-end securing of the platform (robust authentication, session and device management, role segregation, compliance) and its Cloud/DevOps industrialisation (modular containerised architecture, attack-surface reduction, controlled deployment).

Keywords: cybersecurity, authentication, JWT, session management, access control, GDPR, Cloud Computing, DevOps, containerisation, software architecture, NestJS, project management.

Table des matières

Rapport de soutenance de fin d'alternance.....	1
Remerciements.....	2
Résumé.....	2
Abstract.....	3
Table des matières.....	4
Glossaire et acronymes.....	5
Introduction.....	6
Contexte de l'alternance.....	8
1.1 L'entreprise d'accueil : Bioénergie.....	8
1.2 L'environnement et l'organisation de travail.....	8
1.3 La mission confiée et mon positionnement.....	9
Partie 2 - Le projet CarPulse : problématique et cadrage.....	10
2.1 Le besoin métier.....	10
2.2 Objectifs, contraintes et exigences non fonctionnelles.....	11
2.3 Architecture globale.....	12
Partie 3 - La réalisation technique.....	13
3.1 La collecte de données.....	13
3.2 L'analyse et le scoring.....	14
3.3 Le backend d'API.....	15
3.4 Le frontend et le site vitrine.....	16
Partie 4 - Le fil rouge cybersécurité.....	16
4.1 Sécurité de l'authentification.....	16
4.1.1 Protection des mots de passe.....	17
4.1.2 Jetons JWT : access token et refresh token.....	17
4.1.3 Stockage des jetons : jamais en clair.....	17
4.1.4 Rotation des jetons et détection de réutilisation.....	17
4.1.5 Fédération d'identité et parcours de compte.....	18
4.2 Gestion des sessions et des appareils.....	18
4.3 Contrôle d'accès et cloisonnement des rôles.....	19
4.4 Sécurité applicative et défense en profondeur.....	19
4.5 Sécurité de la collecte de données : enjeux juridiques, éthiques et RGPD.....	20
4.6 Analyse de risques et pistes d'amélioration.....	21
Partie 5 - Cloud et DevOps.....	22
5.1 Une architecture pensée pour le Cloud.....	22
5.2 Conteneurisation.....	22
5.3 Configuration et gestion des secrets (approche 12 facteurs).....	23

5.4 Déploiement et exploitation.....	23
5.5 Sécurité de l'infrastructure et accès distant.....	24
5.6 Maturité DevOps et perspectives d'industrialisation.....	24
Partie 6 - Bilan de l'alternance.....	25
6.1 Compétences acquises et développées.....	25
6.2 Difficultés rencontrées et solutions apportées.....	25
6.3 Apports pour l'entreprise et perspectives.....	26
Conclusion.....	26
Bibliographie et webographie.....	27
Annexes.....	28

Introduction

Le marché de l'occasion automobile se caractérise par une forte volatilité des prix et une asymétrie d'information entre vendeurs et acheteurs. Identifier une « bonne affaire » suppose de comparer, en continu et à grande échelle, un très grand nombre d'annonces au regard de leurs caractéristiques (modèle, âge, kilométrage, motorisation, équipements). C'est de ce constat qu'est né **CarPulse**, le projet sur lequel j'ai travaillé durant mon alternance chez **Bioénergie** et qui constitue le sujet de cette soutenance.

CarPulse est une plateforme complète qui automatise trois étapes : la **collecte** de données de marché, leur **analyse** par plusieurs méthodes de valorisation, et leur **restitution** via une application web à destination d'utilisateurs authentifiés. Derrière cette apparente simplicité fonctionnelle se cache une architecture distribuée mobilisant des technologies variées - collecte en Python, base de données PostgreSQL, API en NestJS, interface en Next.js, le tout conteneurisé - et surtout un ensemble d'**exigences de sécurité et d'industrialisation** que ma formation m'a conduit à placer au centre de mes préoccupations.

Au sein de l'équipe, j'ai occupé un rôle transverse : **chef de projet, analyste fonctionnel et product manager**, ce qui m'a donné une vision d'ensemble du produit et de sa trajectoire, les autres membres de l'équipe me faisant remonter régulièrement leurs avancements. Parallèlement, ma contribution technique

s'est concentrée sur les deux dimensions au cœur de mon master : le **backend d'authentification et la sécurisation applicative**, ainsi que la **démarche DevOps et l'industrialisation Cloud** de l'ensemble de la plateforme (spécialité secondaire *Cloud Computing*).

Ce positionnement - pilotage global, expertise sécurité et maîtrise du cycle DevOps - structure la problématique de ce rapport :

Comment concevoir, sécuriser et industrialiser une plateforme d'intelligence de marché manipulant des données massives et des comptes utilisateurs, tout en garantissant sa conformité, sa maintenabilité et un déploiement maîtrisé ?

Pour y répondre, ce rapport s'organise en six parties. La **première** présente le contexte de l'alternance : l'entreprise Bioénergie, l'environnement de travail et la mission qui m'a été confiée. La **deuxième** cadre le projet CarPulse : le besoin métier, les objectifs et l'architecture globale. La **troisième** décrit la réalisation technique des différentes briques. La **quatrième**, premier cœur du rapport, développe le **fil rouge cybersécurité** : authentification, sessions, contrôle d'accès, conformité et défense en profondeur. La **cinquième** développe le second fil rouge, **Cloud & DevOps** : conteneurisation, isolation, déploiement et sécurité de l'infrastructure. Enfin, la **sixième** dresse un bilan des compétences acquises, des difficultés rencontrées et des perspectives.

I - Contexte de l'alternance

1.1 L'entreprise d'accueil : Bioénergie

J'ai réalisé mon alternance au sein de l'entreprise **Bioénergie**, une structure à taille humaine dont l'activité s'articule autour de deux pôles : les **énergies renouvelables** d'une part, et l'**achat-revente de véhicules automobiles** d'autre part.

- **Secteur d'activité** : énergies renouvelables et négoce automobile (achat-revente de véhicules).
- **Effectif** : 12 collaborateurs.

- **Implantation** : France - siège à Strasbourg.
- **Modalité de travail** : j'ai exercé ma mission **en télétravail intégral depuis Abomey-Calavi (Bénin)**, ce qui a constitué une contrainte structurante du projet, tant sur le plan de la collaboration à distance que sur celui de l'accès sécurisé aux environnements de travail.

C'est précisément l'activité d'**achat-revente automobile** de Bioénergie qui a fait naître le besoin à l'origine de CarPulse : disposer d'un outil capable d'analyser en continu le marché de l'occasion afin d'**identifier les véhicules sous-évalués** représentant une opportunité d'achat. Le projet s'inscrit ainsi directement dans la chaîne de valeur de l'entreprise, comme un outil d'aide à la décision destiné à objectiver et à accélérer le repérage des bonnes affaires. La plateforme est accessible publiquement à l'adresse <https://getcarpulse.com/>.

Note de rédaction : ce paragraphe reflète les informations que tu m'as transmises. Si tu souhaites détailler la forme juridique (SARL, SAS...), l'ancienneté ou le chiffre d'affaires de Bioénergie, indique-le-moi et je complète.

1.2 L'environnement et l'organisation de travail

Mon intégration s'est faite en qualité d'**ingénieur logiciel** au sein de l'équipe technique de Bioénergie, dans un environnement de travail entièrement distant (télétravail depuis le Bénin, entreprise basée en France). Cette configuration a rendu d'autant plus déterminantes la rigueur d'organisation et la qualité des outils collaboratifs. L'environnement s'est structuré autour des pratiques suivantes :

- **Méthodologie de gestion de projet** : approche itérative de type agile, avec des points d'avancement réguliers menés à distance. En tant que chef de projet, j'animais ces points et centralisais les remontées des développeurs data, backend et frontend.
- **Outils de collaboration et de versionnement** : gestion du code par dépôts Git distincts pour chaque brique (Data, Backend, Frontend, Landing), favorisant une séparation nette des responsabilités.
- **Chaîne technique** : environnements de développement conteneurisés (Docker), bases de données PostgreSQL, et outillage qualité (tests automatisés, *linters*, formatage automatique).

Cette organisation multi-briques, avec un dépôt par composant, reflète une architecture volontairement **modulaire** : chaque partie du système peut évoluer

et être déployée indépendamment, ce qui a des vertus tant en matière de maintenabilité que de **réduction de la surface d'attaque** (isolation des responsabilités) - un principe que je développe dans les parties consacrées à la sécurité et au Cloud.

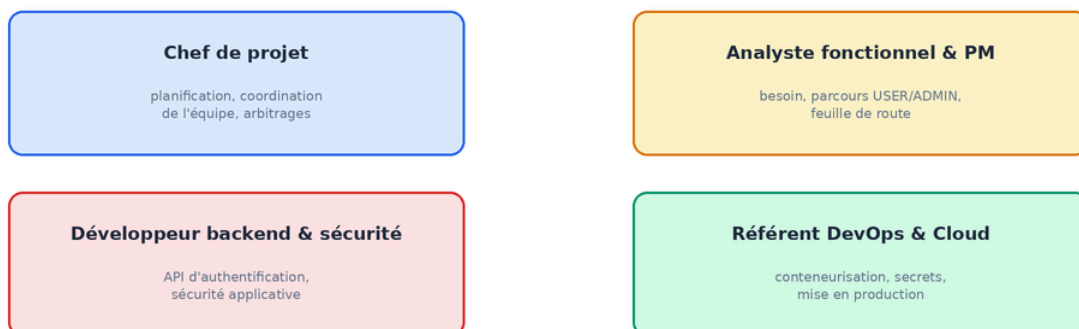
1.3 La mission confiée et mon positionnement

La mission qui m'a été confiée dépassait le simple développement : il s'agissait de **piloter la conception d'une plateforme complète**, d'en **garantir la sécurité** et d'en **industrialiser le déploiement**. Concrètement, mon rôle s'est décliné sur quatre plans complémentaires :

1. **Chef de projet** - planification, coordination de l'équipe, arbitrage des priorités et suivi de l'avancement des briques data, backend et frontend.
2. **Analyste fonctionnel & product manager** - recueil et formalisation du besoin, définition des parcours utilisateurs (notamment les univers *USER* et *ADMIN*), rédaction des spécifications et priorisation de la feuille de route produit.
3. **Développeur backend & référent sécurité** - conception et développement de l'API d'authentification et de gestion des comptes, ainsi que la définition des mesures de sécurité transverses à la plateforme.
4. **Référent DevOps & Cloud** - conteneurisation des services, définition de l'architecture de déploiement, isolation réseau des composants et industrialisation du cycle de vie applicatif (spécialité secondaire *Cloud Computing*).

Un rôle transverse : quatre casquettes

Une vision complète du produit, du besoin jusqu'au déploiement



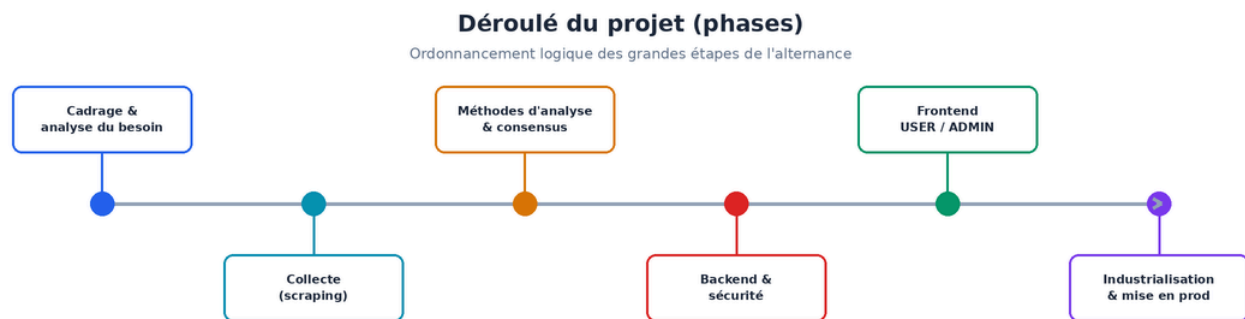
Sécurité intégrée dès la conception (security by design), à tous les niveaux

Ce positionnement transverse constitue, à mon sens, la principale valeur de cette alternance au regard de mon master : il m'a permis de ne pas cantonner la sécurité à une étape technique isolée, mais de l'intégrer **dès la conception** et à **tous les niveaux** du projet - depuis la définition du besoin jusqu'au déploiement de l'infrastructure.

Les objectifs assignés au projet étaient les suivants :

- Disposer d'une source de données de marché **fiable et régulièrement actualisée** ;
- Produire une évaluation **objective et explicable** de la qualité d'une offre ;
- Offrir un accès **sécurisé et différencié par rôle** aux résultats ;
- Garantir la **conformité** de la collecte et du traitement des données ;
- Assurer un **déploiement reproductible et maîtrisé** de l'ensemble des services.

Ces objectifs servent de fil directeur aux parties suivantes.



II - Le projet CarPulse : problématique et cadrage

2.1 Le besoin métier

L'activité d'achat-revente automobile de Bioénergie repose sur une compétence centrale : savoir repérer, parmi des milliers d'annonces, les véhicules dont le prix affiché est inférieur à leur valeur réelle de marché. Réalisé manuellement, ce

travail de veille est chronophage, subjectif et difficilement reproductible : un acheteur ne peut consulter qu'un nombre limité d'annonces, et son jugement dépend de son expérience et de sa disponibilité.

Le besoin exprimé était donc le suivant : **disposer d'un outil automatisé, objectif et permanent, capable d'évaluer la qualité d'une offre automobile et de faire remonter les meilleures opportunités.** En tant qu'analyste fonctionnel, j'ai traduit ce besoin en trois exigences fonctionnelles :

1. **Collecter** en continu les annonces disponibles sur une place de marché de référence, avec leurs caractéristiques détaillées ;
2. **Évaluer** chaque annonce de façon fiable, en produisant un indicateur clair de « bonne affaire » assorti d'un niveau de confiance ;
3. **Restituer** ces résultats dans une interface accessible, filtrable et sécurisée, distinguant les usages internes (administration) des usages de consultation.
- 4.

2.2 Objectifs, contraintes et exigences non fonctionnelles

Au-delà des fonctionnalités, j'ai formalisé un ensemble d'exigences non fonctionnelles qui ont fortement orienté les choix techniques :

Exigence	Description	Impact sur la conception
Fiabilité de l'évaluation	Un mauvais classement peut induire une décision d'achat coûteuse.	Recours à trois méthodes d'analyse indépendantes et à un vote de consensus, plutôt qu'à un seul modèle.
Sécurité	La plateforme gère des comptes, des données de marché à valeur concurrentielle et un espace d'administration.	Authentification forte, cloisonnement des rôles, défense en profondeur (Partie 4).
Conformité	La collecte de données pose des questions juridiques et de protection des données.	Analyse de licéité et démarche RGPD (Partie 4).

Exigence	Description	Impact sur la conception
Maintenabilité	Le produit doit pouvoir évoluer sur la durée.	Architecture modulaire, <i>Domain-Driven Design</i> , tests automatisés.
Reproductibilité du déploiement	Le projet est développé à distance (Bénin) pour une entreprise en France.	Conteneurisation intégrale et industrialisation DevOps (Partie 5).

La contrainte de **télétravail international** mérite d'être soulignée : développer et opérer une plateforme distribuée depuis le Bénin pour une entreprise strasbourgeoise a imposé une discipline particulière en matière d'accès distant sécurisé, de gestion des secrets et de reproductibilité des environnements - autant de préoccupations relevant directement de mes deux spécialités, cybersécurité et Cloud.

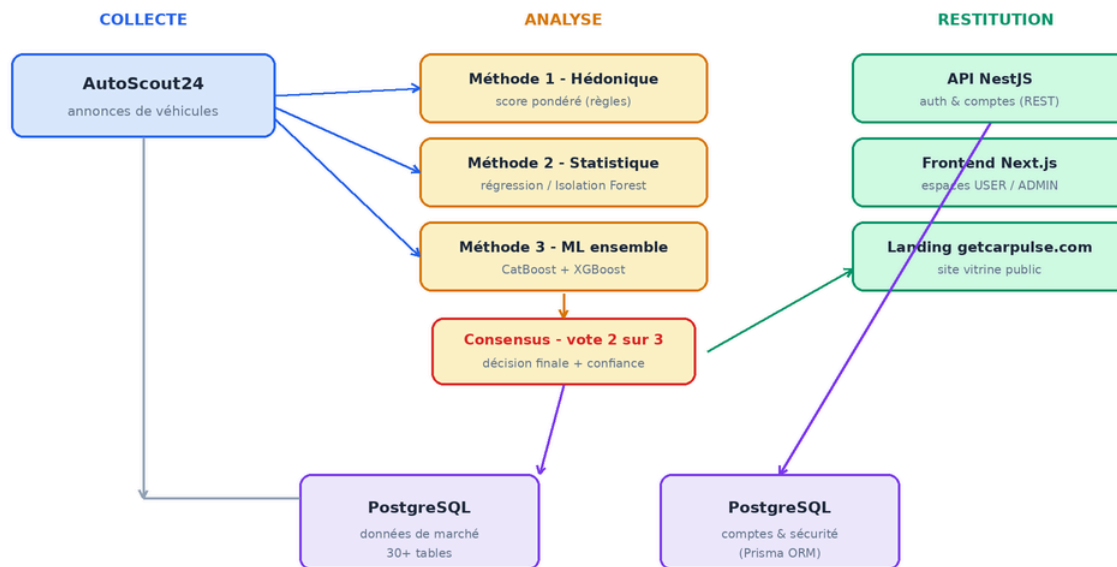
2.3 Architecture globale

Pour répondre à ces exigences, CarPulse a été conçu comme un ensemble de **briques faiblement couplées**, chacune spécialisée dans une responsabilité et versionnée dans un dépôt distinct. On distingue trois grands domaines : la **collecte et l'analyse de données** (brique Data), les **services applicatifs** (backend d'API et base de données des comptes), et la **restitution** (frontend applicatif et site vitrine).

Le schéma suivant résume le flux global, de la collecte jusqu'à la restitution :

Annexe A - Architecture globale de la plateforme CarPulse

Collecte -> Analyse (3 méthodes + consensus) -> Restitution



Deux bases de données PostgreSQL coexistent, avec des responsabilités distinctes :

- une base « **données de marché** », alimentée par la collecte et exploitée par les méthodes d'analyse (annonces, métriques d'analyse, résultats de consensus) ;
- une base « **comptes et sécurité** », gérée par le backend NestJS via l'ORM Prisma (utilisateurs, sessions, appareils, jetons).

Cette séparation n'est pas seulement organisationnelle : elle participe de la **défense en profondeur**, en isolant les données personnelles des utilisateurs des données de marché, et en limitant la portée d'une éventuelle compromission d'un composant.

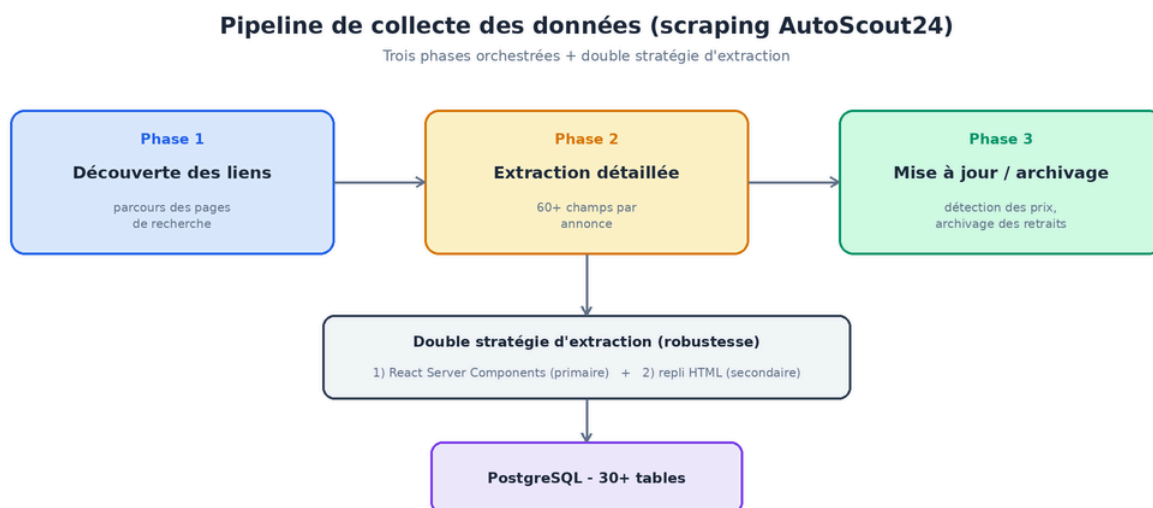
Les trois briques principales sont détaillées dans la partie suivante.

III - La réalisation technique

Cette partie décrit les trois briques de la plateforme. J'y adopte une posture de chef de projet et d'architecte : je présente les choix structurants et les

responsabilités de chaque composant, avant d’approfondir dans les parties suivantes les dimensions qui relèvent directement de mes spécialités (sécurité et Cloud).

3.1 La collecte de données



La brique **CarPulse-Data** (Python) alimente l’ensemble du système. Elle collecte les annonces d’une place de marché automobile de référence, en trois phases orchestrées :

1. **Découverte des liens** - parcours des pages de recherche pour extraire les URL des annonces ;
2. **Extraction détaillée** - récupération, pour chaque annonce, de plus de 60 champs : prix, kilométrage, marque et modèle, année, puissance, énergie, boîte de vitesses, carrosserie, équipements, consommation, émissions de CO2, informations vendeur, etc. ;
3. **Mise à jour et archivage** - re-vérification périodique des annonces existantes pour détecter les changements de prix et archiver les annonces retirées.

Deux stratégies d’extraction sont combinées pour la robustesse : l’analyse des **React Server Components** (données structurées injectées par le site), avec un repli sur une extraction HTML classique lorsque nécessaire. La collecte s’appuie sur un pilotage de navigateur en mode furtif afin de franchir les protections

anti-robot, ce qui soulève des questions techniques mais aussi **juridiques et éthiques** que je traite spécifiquement en Partie 4.

L'ensemble alimente une base **PostgreSQL** organisée en une trentaine de tables (annonces, historique de prix, métriques d'analyse, résultats de consensus). Un tableau de bord de supervision permet de suivre en temps réel l'avancement et la santé de la collecte.

3.2 L'analyse et le scoring

Le cœur de la valeur ajoutée de CarPulse réside dans sa capacité à qualifier une annonce. Plutôt que de faire reposer cette décision sur un modèle unique - forcément faillible - nous avons retenu une approche à **trois méthodes indépendantes**, dont les verdicts sont ensuite agrégés. Chaque méthode produit, pour chaque annonce, un booléen « bonne affaire », un score et un niveau de confiance.

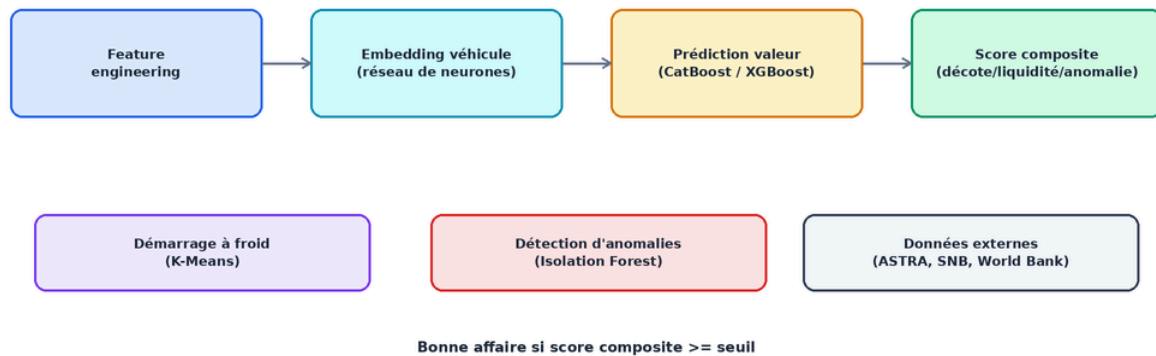
Méthode 1 - Scoring hédonique (à base de règles). Un score composite pondéré sur cinq dimensions : le positionnement prix (40 %), l'attractivité du véhicule (25 %), la confiance vendeur (15 %), la qualité de l'annonce (10 %) et la rareté (10 %). Une annonce est jugée intéressante au-delà d'un seuil configurable.

Méthode 2 - Positionnement statistique. Une analyse statistique à deux niveaux : lorsque l'échantillon de véhicules comparables est suffisant, une régression de tendance est appliquée ; sinon, une détection d'anomalies par *Isolation Forest* prend le relais. Le résultat classe l'annonce selon des paliers (exceptionnelle, bonne, correcte, surévaluée).

Méthode 3 - Ensemble de *machine learning*. Une chaîne complète de prédiction de la valeur résiduelle : plongement dense des couples marque/modèle par réseau de neurones, prédiction par modèles à gradient boosté (CatBoost, XGBoost) sous contraintes de monotonie, gestion du démarrage à froid par regroupement (*K-Means*) et détection d'anomalies. Le score composite combine décote, liquidité et anomalie.

Méthode 3 - Chaîne de machine learning (valeur résiduelle)

Prédiction de la valeur + score composite



Consensus - vote majoritaire 2-sur-3. La décision finale résulte d'un vote : une annonce est retenue comme bonne affaire si au moins deux des trois méthodes l'approuvent. Le système qualifie également le niveau d'accord (unanime, majoritaire, partagé), ce qui renforce l'explicabilité et la confiance dans le résultat - un point important lorsqu'une décision d'achat en dépend.

Cette architecture d'analyse illustre un principe cher à la sécurité comme à la science des données : la **redondance** et la **diversité** des sources de décision réduisent le risque d'erreur systématique.

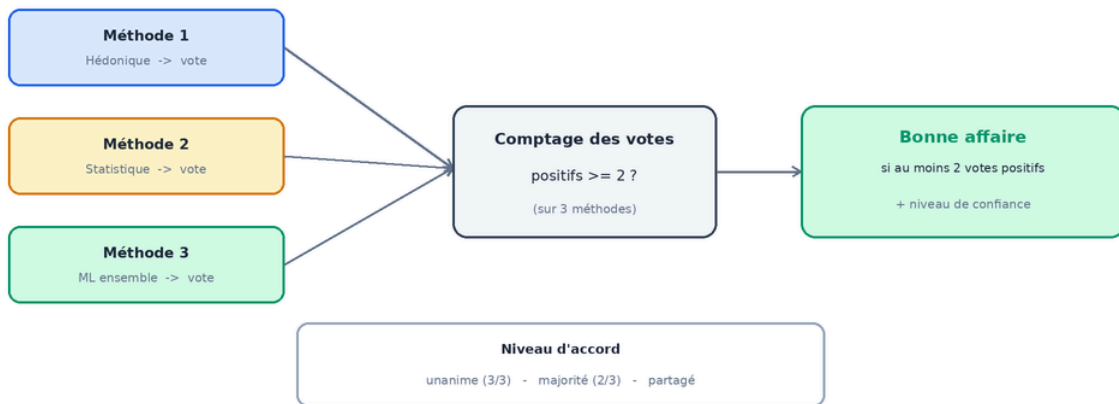
3.3 Le backend d'API

La brique **CarPulse-Backend** est le composant que j'ai personnellement conçu et développé. Il s'agit d'une API bâtie avec le framework **NestJS** (Node.js / TypeScript), adossée à **PostgreSQL** via l'ORM **Prisma**.

L'architecture suit les principes du **Domain-Driven Design** : chaque module métier est découpé en quatre couches nettement séparées - domain (entités et règles métier), app (services applicatifs et DTO), http (contrôleurs exposant les routes) et infra (implémentations techniques, notamment les dépôts Prisma).

Décision par consensus : vote majoritaire 2 sur 3

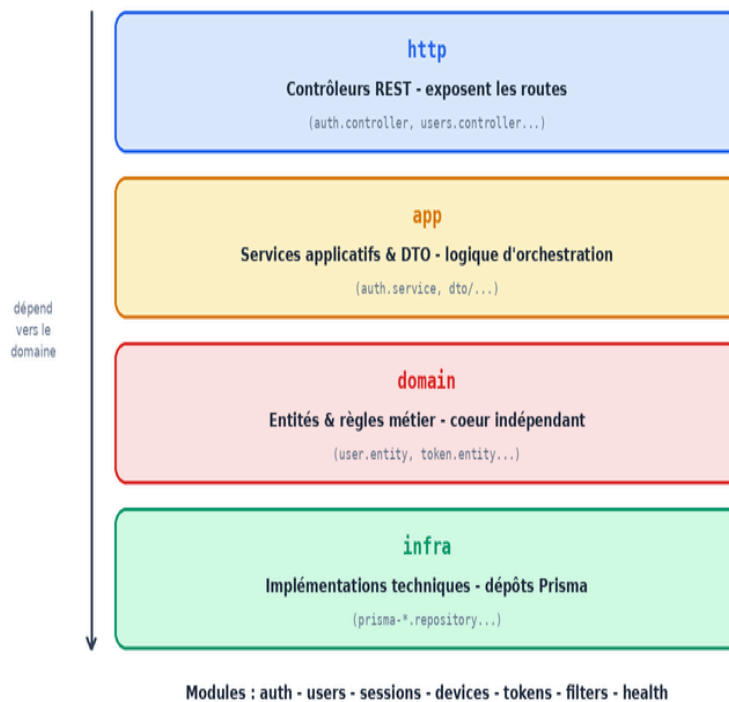
Trois méthodes indépendantes -> un verdict fiable et explicable



Cette rigueur structurelle facilite les tests, la relecture et l'évolution, et elle matérialise une séparation claire des responsabilités.

Architecture en couches (Domain-Driven Design)

Chaque module backend est découpé en 4 couches

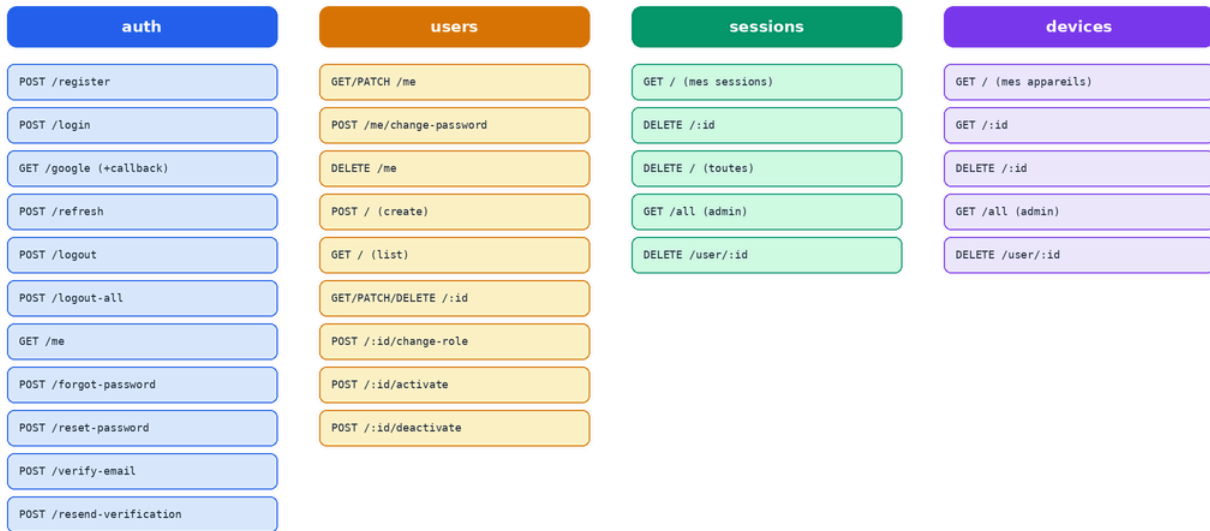


Les modules développés sont les suivants :

Module	Responsabilité
auth	Inscription, connexion, OAuth Google, rafraîchissement de jeton, déconnexion (simple et globale), vérification d'e-mail, mot de passe oublié / réinitialisation.
users	Gestion des comptes (CRUD), changement de mot de passe, activation/désactivation, changement de rôle.
sessions	Liste et révocation des sessions actives d'un utilisateur (y compris révocation globale).
devices	Suivi et gestion des appareils associés à un compte.
tokens	Cycle de vie des jetons (émission, hachage, expiration, révocation).
filters	Gestion des filtres de recherche personnalisés des utilisateurs.
health	Points de contrôle de disponibilité du service (supervision).

Ces modules exposent une API REST complète (inscription, authentification, gestion du profil me, administration des utilisateurs, gestion des sessions et des appareils). Les mécanismes de sécurité qui les sous-tendent - hachage des mots de passe, jetons JWT à double niveau, rotation et révocation, cloisonnement des rôles - font l'objet de la Partie 4.

Cartographie de l'API REST par module



+ module health (points de contrôle de disponibilité)

3.4 Le frontend et le site vitrine

La brique **CarPulse-Frontend** est l'application web de consultation et d'administration, développée avec **Next.js 16** et **React 19** en TypeScript, avec **TanStack Query** pour la gestion des données serveur et **Tailwind CSS** pour la mise en forme.

Sa caractéristique architecturale majeure est la prise en charge de **deux univers distincts** : un espace **USER** (consultation des bonnes affaires, activité, filtres personnalisés) et un espace **ADMIN** (tableau de bord, gestion des utilisateurs, analytique). Cette séparation est stricte dès le routage et les *layouts*, et elle est protégée par un **middleware** qui vérifie le rôle de l'utilisateur avant chaque accès et le redirige le cas échéant. La qualité est soutenue par **Storybook** (documentation des composants) et **Vitest** (tests).

Enfin, la **landing page** (site vitrine getcarpulse.com), également en Next.js, assure la présentation publique du produit. Elle est volontairement isolée de l'application afin de séparer l'exposition publique des fonctionnalités authentifiées - une nouvelle illustration du principe de réduction de la surface d'attaque.

IV - Le fil rouge cybersécurité

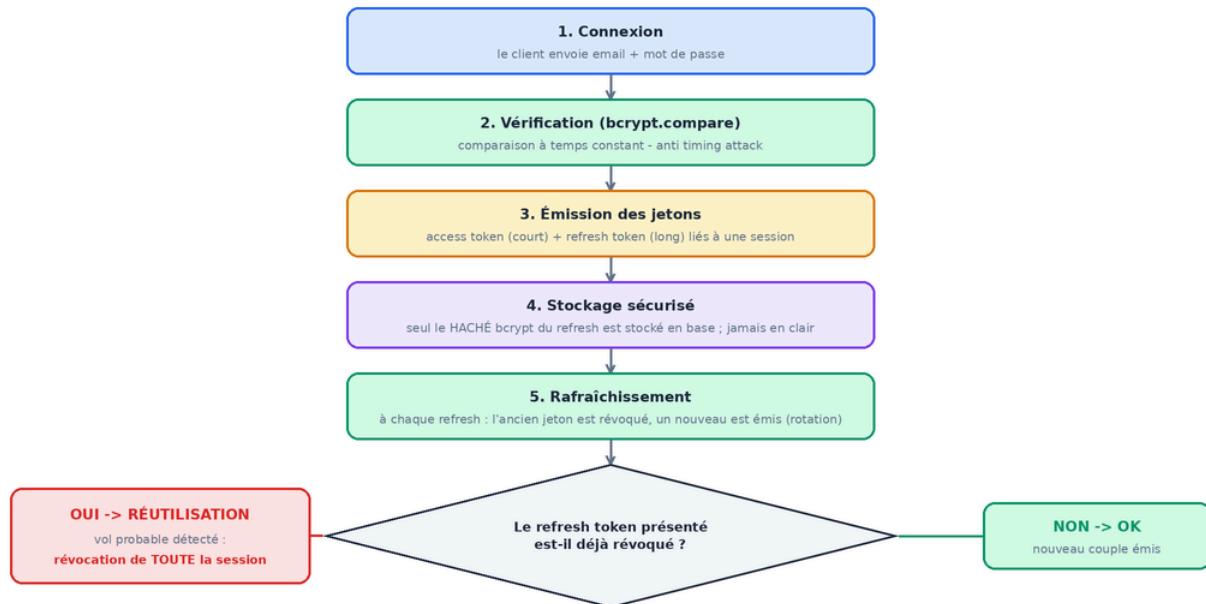
Cette partie constitue le cœur de mon rapport, en cohérence avec ma spécialité principale. J'y détaille les mécanismes de sécurité que j'ai conçus et implémentés dans le backend, puis les enjeux de conformité liés à la collecte de données, et enfin une analyse de risques structurée. L'ensemble illustre une démarche de **sécurité par conception** (*security by design*) : la sécurité n'a pas été ajoutée après coup, mais pensée dès l'architecture.

4.1 Sécurité de l'authentification

L'authentification est la porte d'entrée de la plateforme et, à ce titre, la première cible d'un attaquant. J'ai retenu un modèle fondé sur des jetons **JWT à double niveau**, avec des garanties fortes sur le stockage et le cycle de vie des secrets.

Annexe B - Flux d'authentification : rotation des jetons & détection de réutilisation

JWT access (court) + refresh (long, haché en base) - conforme aux recommandations OWASP



Un jeton volé devient inexploitable : sa réutilisation déclenche la révocation de la session entière.
L'utilisateur peut aussi lister et révoquer ses sessions / appareils actifs (déconnexion globale).

4.1.1 Protection des mots de passe

Les mots de passe ne sont jamais stockés en clair : ils sont hachés avec **bcrypt**, un algorithme volontairement lent et paramétré par un facteur de coût, ce qui rend les attaques par force brute et par dictionnaire coûteuses. La vérification à la connexion s'appuie sur `bcrypt.compare()`, qui effectue une **comparaison à temps constant** : le temps de réponse ne dépend pas de l'endroit où deux valeurs diffèrent, ce qui neutralise les attaques par analyse temporelle (*timing attacks*).

4.1.2 Jetons JWT : access token et refresh token

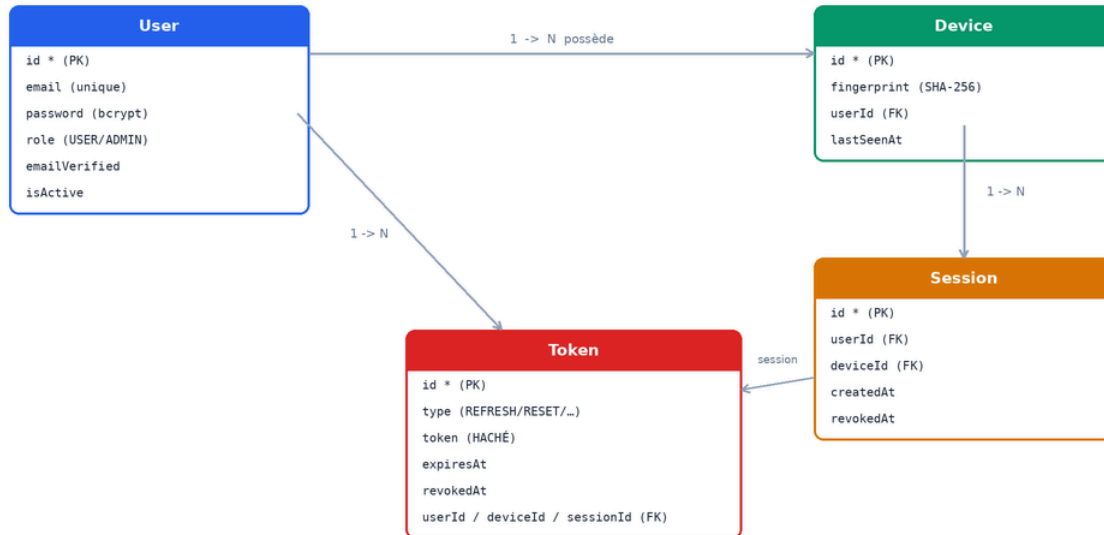
Deux types de jetons sont émis à la connexion :

- un **access token** de courte durée de vie, présenté à chaque requête pour prouver l'identité ; il transporte l'identifiant de l'utilisateur, son e-mail, son **rôle** et l'identifiant de session ;
- un **refresh token** de durée de vie plus longue, dont l'unique fonction est d'obtenir un nouvel access token sans redemander les identifiants.

La courte durée de vie de l'access token limite la fenêtre d'exploitation en cas de vol : un jeton dérobé expire rapidement.

Annexe C - Modèle de données (base comptes & sécurité, via Prisma)

Un utilisateur possède des appareils, des sessions et des jetons révocables



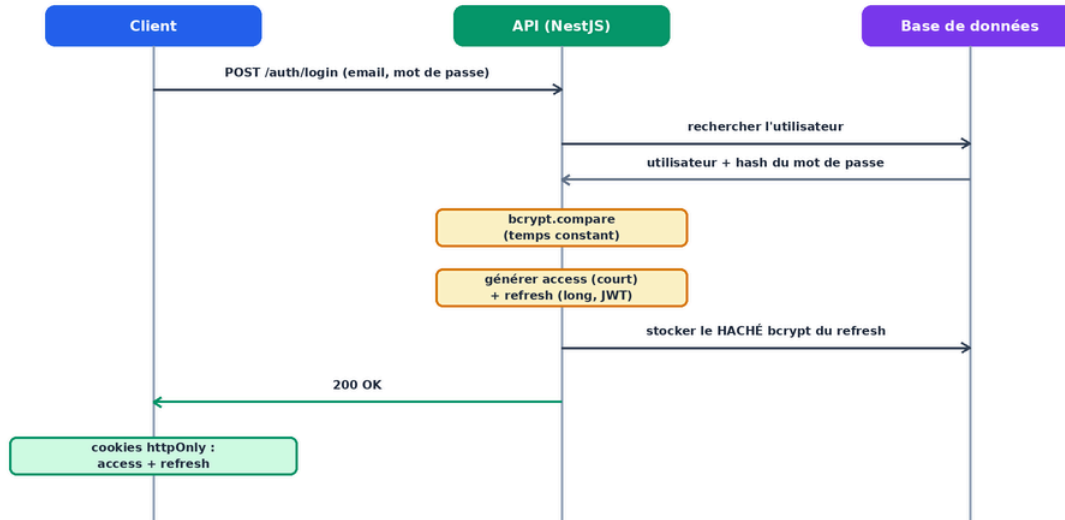
Chaque jeton (token) référence l'utilisateur, l'appareil et la session : révocation ciblée ou globale possible.

4.1.3 Stockage des jetons

Point essentiel de la conception : **le refresh token n'est jamais stocké en clair côté serveur**. Seul le client détient le jeton en clair ; la base de données ne conserve qu'un **haché bcrypt** (facteur de coût 12) du refresh token. Ainsi, même en cas de fuite de la base de données, un attaquant ne peut pas rejouer les jetons volés. Le même principe s'applique aux jetons à usage unique : les jetons de **réinitialisation de mot de passe** et de **vérification d'e-mail** sont eux aussi hachés avant stockage.

Diagramme de séquence : connexion (login)

Vérification, émission des jetons et cookies sécurisés

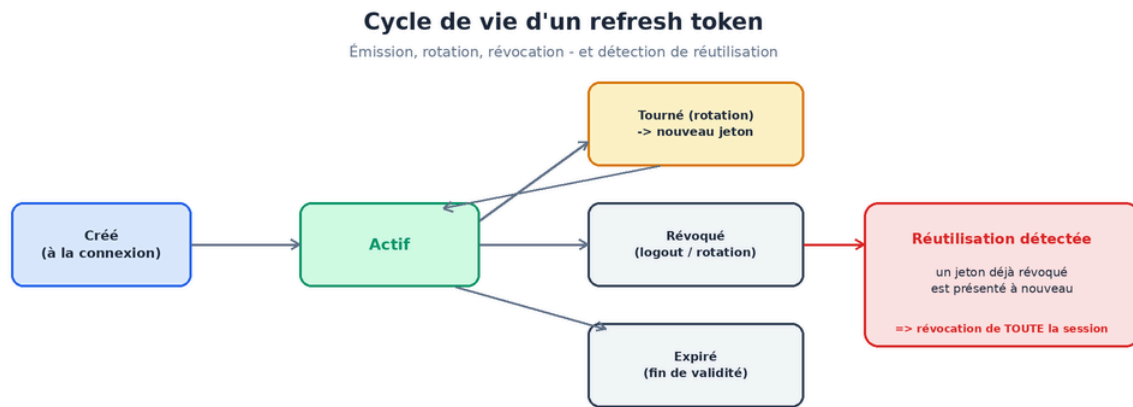


4.1.4 Rotation des jetons et détection de réutilisation

Le mécanisme dont je suis le plus satisfait est la **rotation des refresh tokens avec détection de réutilisation**, recommandation de référence de l'OWASP pour les applications à jetons. Son fonctionnement :

1. à chaque rafraîchissement, l'ancien refresh token est **révoqué** et un nouveau est émis (rotation obligatoire) ;
2. si un refresh token **déjà utilisé** (donc révoqué) est présenté à nouveau, le système en déduit qu'il a probablement été volé et **révoque immédiatement l'intégralité de la session** concernée, journalisant l'événement comme une atteinte de sécurité (« *token reuse* »).

Concrètement, si un attaquant parvient à voler un refresh token, sa première utilisation par la victime ou par l'attaquant invalidera la chaîne pour l'autre partie, et la réutilisation déclenchera la révocation totale. Ce dispositif transforme un vol de jeton silencieux en incident détectable et contenu. Un contrôle complémentaire vérifie qu'il n'existe **qu'un seul jeton actif par session** après rotation, afin de repérer toute anomalie.



Rotation obligatoire : un seul jeton actif par session. Un jeton volé devient inexploitable.

4.1.5 Fédération d'identité et parcours de compte

Outre l'authentification par identifiants, j'ai intégré une **connexion via Google (OAuth 2.0)**, qui délègue la preuve d'identité à un fournisseur de confiance et évite de gérer un mot de passe supplémentaire. Le parcours de compte complet a été sécurisé : **vérification d'e-mail, mot de passe oublié / réinitialisation** par jeton haché à usage unique et à durée limitée, **changement de mot de passe** et **déconnexion globale** (révocation de toutes les sessions d'un utilisateur, utile en cas de suspicion de compromission).

4.2 Gestion des sessions et des appareils

Au-delà du couple de jetons, j'ai modélisé explicitement les notions de **session** et d'**appareil**, ce qui apporte une traçabilité et un pouvoir de révocation précieux du point de vue défensif.

- Chaque connexion est rattachée à un **appareil**, identifié par une **empreinte** calculée (hachage SHA-256 d'une combinaison de caractéristiques de la requête). Cela permet de reconnaître les appareils connus et de repérer les connexions inhabituelles.
- L'utilisateur peut **lister ses sessions et appareils actifs** et en **révoquer** un individuellement, ou tous d'un coup. C'est la contre-mesure directe au **vol de session** : un utilisateur qui suspecte une compromission peut invalider un accès sans changer tout son environnement.

- Côté administration, il est possible de révoquer les sessions d'un utilisateur donné, ce qui offre un levier de réponse à incident.

Cette gestion fine matérialise un principe de sécurité fondamental : la capacité à **révoquer** un accès est aussi importante que la capacité à l'**accorder**.

4.3 Contrôle d'accès et cloisonnement des rôles

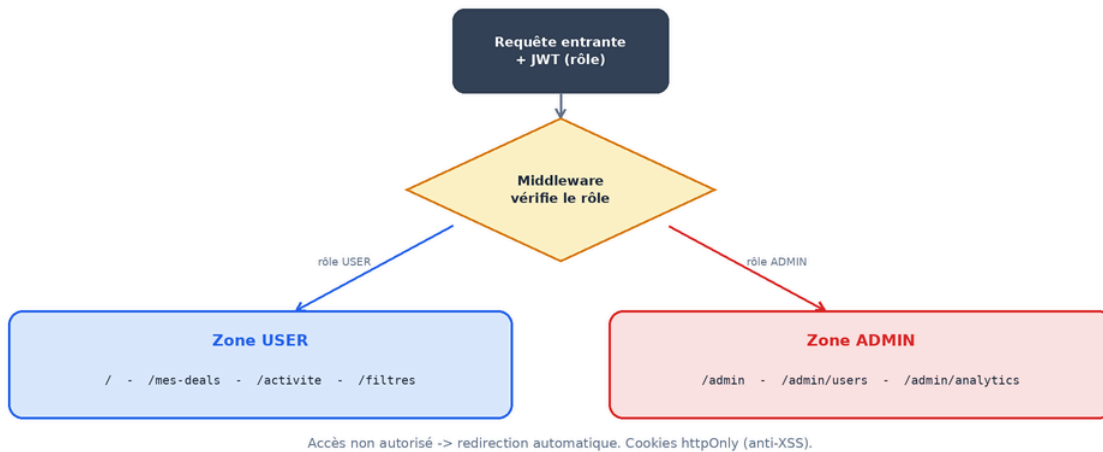
La plateforme distingue deux rôles, **USER** et **ADMIN**, et applique un **contrôle d'accès fondé sur les rôles** (RBAC) à plusieurs niveaux, selon le principe de défense en profondeur :

- **Au niveau du jeton** : le rôle est inscrit dans l'access token et vérifié à chaque requête par la stratégie JWT.
- **Au niveau de l'API** : un RolesGuard protège les routes réservées aux administrateurs (par exemple la gestion des utilisateurs, le changement de rôle, l'activation/désactivation de comptes).
- **Au niveau de l'objet** : un OwnershipGuard vérifie qu'un utilisateur n'accède qu'à ses propres ressources. C'est la protection directe contre les failles d'**autorisation au niveau des objets** (*Broken Object Level Authorization*, en tête du classement OWASP API Security), où un utilisateur tenterait d'accéder aux données d'un autre en modifiant un identifiant dans l'URL.
- **Au niveau du frontend** : un **middleware** cloisonne strictement les univers USER et ADMIN dès le routage et redirige tout accès non autorisé.

Enfin, côté navigateur, les jetons sont véhiculés par des **cookies httpOnly**, inaccessibles au JavaScript, ce qui réduit fortement l'impact d'une éventuelle faille de type **XSS** (un script injecté ne peut pas lire le jeton).

Cloisonnement des rôles : middleware de protection

Séparation stricte des univers USER et ADMIN dès le routage

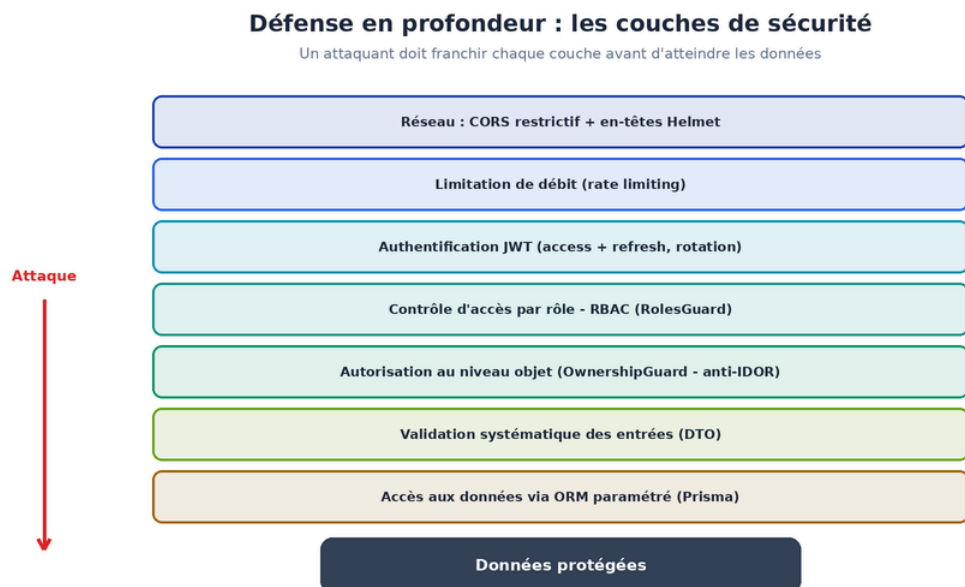


4.4 Sécurité applicative et défense en profondeur

Plusieurs protections transverses ont été mises en place au niveau de l'API :

Mesure	Rôle défensif
Helmet	Positionne des en-têtes HTTP de sécurité (protection contre le <i>clickjacking</i> , le <i>sniffing</i> de type MIME, etc.).
CORS restrictif	N'autorise que les origines, méthodes et en-têtes explicitement déclarés, avec gestion des <i>credentials</i> .
Validation systématique des entrées	Un <i>ValidationPipe</i> global valide et filtre les données entrantes via des DTO typés, écartant les champs non attendus (défense contre l'injection et la sur-affectation).
Limitation de débit (<i>rate limiting</i>)	Un <i>throttler</i> limite le nombre de requêtes, y compris derrière un proxy inverse (prise en compte correcte de l'adresse réelle du client), contre la force brute et le déni de service.
Filtres d'exception centralisés	Uniformisent les réponses d'erreur et évitent les fuites d'informations techniques sensibles.

Mesure	Rôle défensif
Séparation des bases de données	Les données personnelles (comptes) sont isolées des données de marché, limitant la portée d'une compromission.



Cette accumulation de couches indépendantes est l'application concrète du principe de **défense en profondeur** : la défaillance d'une mesure ne suffit pas, à elle seule, à compromettre le système.

4.5 Sécurité de la collecte de données : enjeux juridiques, éthiques et RGPD

La brique de collecte soulève des questions de sécurité au sens large - juridique et éthique - que je me dois d'aborder honnêtement dans un rapport de master en cybersécurité.

Licéité de la collecte. Le *scraping* de données publiques se situe dans une zone à encadrer : il convient de respecter les conditions générales d'utilisation de la source, le fichier robots.txt, et d'éviter de porter atteinte au fonctionnement du service (charge maîtrisée). Le recours à un pilote de navigateur en mode furtif pour franchir des protections anti-robot doit être questionné : dans un cadre professionnel, il est nécessaire de documenter la finalité, de privilégier

lorsqu'elles existent des voies licites (API officielle, accord commercial) et de limiter la collecte au strict nécessaire.

Données personnelles et RGPD. Certaines données collectées peuvent constituer des **données à caractère personnel** (coordonnées de vendeurs particuliers, par exemple). Le traitement doit alors respecter les principes du RGPD : **base légale** (intérêt légitime, à documenter et à mettre en balance), **minimisation** (ne collecter que ce qui est utile à l'évaluation d'une offre), **limitation de conservation**, et information des personnes concernées. Une piste que j'ai identifiée consiste à **pseudonymiser ou exclure** les données personnelles non indispensables au scoring, l'objet du produit étant la valorisation d'un véhicule, non le profilage d'un vendeur.

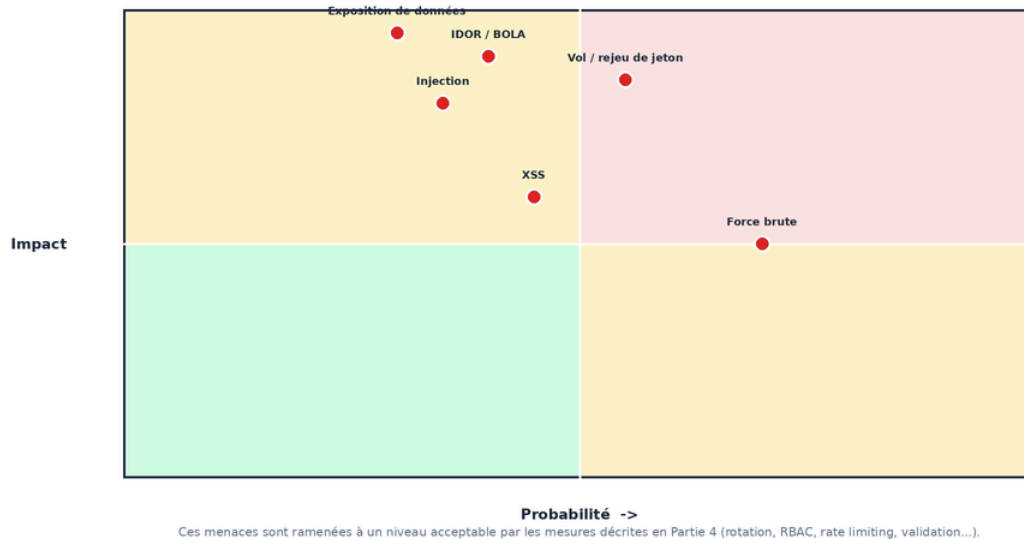
Posture retenue. Sans être juriste, j'ai porté cette réflexion auprès de l'entreprise pour que la démarche soit consciente et documentée, plutôt que subie. Cette lucidité sur les limites juridiques et éthiques d'un dispositif technique fait partie intégrante, à mon sens, du rôle d'un professionnel de la cybersécurité.

4.6 Analyse de risques et pistes d'amélioration

Pour objectiver le niveau de sécurité, j'ai rapproché les principales menaces des mesures en place, en m'appuyant sur le référentiel **OWASP**.

Matrice des risques (positionnement indicatif)

Probabilité x impact, avant mesures de mitigation



Menace (OWASP)	Mesure en place	Amélioration envisagée
Vol de session / rejeu de jeton	Rotation + détection de réutilisation, cookies httpOnly, sessions révocables	Liaison du jeton à l’empreinte d’appareil, alertes de connexion
Force brute sur la connexion	Rate limiting, bcrypt lent	Verrouillage progressif de compte, CAPTCHA, MFA
Autorisation cassée (BOLA/IDOR)	OwnershipGuard, RolesGuard	Tests d’autorisation automatisés systématiques
Injection / entrées malveillantes	ValidationPipe, ORM Prisma (requêtes paramétrées)	Revue de sécurité et <i>fuzzing</i> des endpoints
Exposition de données sensibles	Hachage des secrets, séparation des bases, filtres d’erreur	Chiffrement au repos, rotation régulière des secrets
XSS / en-têtes	Cookies httpOnly, Helmet	Politique de sécurité de contenu (CSP) renforcée

Les pistes d’amélioration prioritaires que je recommande sont l’ajout d’une **authentification multifacteur (MFA)**, la mise en place d’une **journalisation de sécurité centralisée** (traçabilité des événements sensibles), et l’intégration de

tests de sécurité automatisés dans la chaîne d'intégration continue - ce qui fait le lien avec la partie suivante, consacrée au Cloud et au DevOps.

V - Cloud et DevOps

En cohérence avec ma spécialité secondaire, cette partie décrit l'industrialisation de la plateforme : comment CarPulse est packagé, configuré, déployé et exploité. La contrainte de travail à distance (Bénin / France) a fait de ces sujets une nécessité opérationnelle autant qu'un objet d'apprentissage.

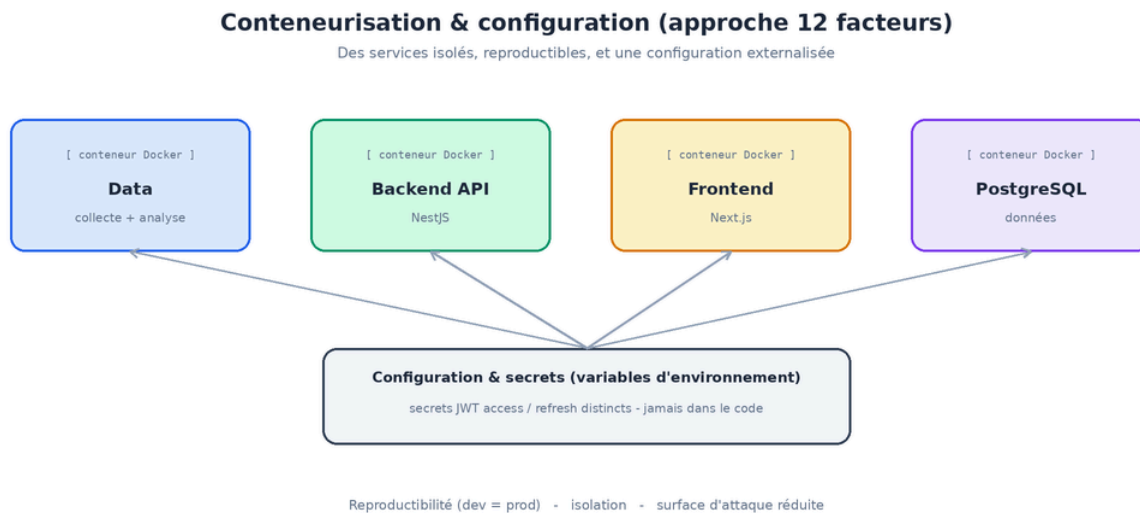
5.1 Une architecture pensée pour le Cloud

Le découpage de CarPulse en briques indépendantes (collecte, analyse, API, frontend, vitrine) n'est pas seulement un choix de conception logicielle : c'est une architecture **orientée services**, adaptée à un déploiement Cloud. Chaque composant :

- possède son propre cycle de vie et peut être déployé, mis à l'échelle et supervisé indépendamment ;
- expose une interface claire (API REST pour le backend, pages web pour le frontend), ce qui autorise des choix d'hébergement différenciés ;
- est isolé des autres, ce qui limite la propagation des pannes comme des compromissions.

Cette modularité est la condition d'une exploitation Cloud maîtrisée : on peut faire évoluer la collecte sans toucher à l'API, ou renforcer l'API sans impacter la vitrine publique.

5.2 Conteneurisation



L'unité de déploiement retenue est le **conteneur Docker**. La conteneurisation garantit que chaque service embarque son environnement d'exécution complet (runtime, dépendances, configuration), ce qui assure la **reproductibilité** : le comportement observé en développement est identique à celui de production. Des fichiers Dockerfile et .dockerignore définissent des images optimisées pour les briques concernées, en n'incluant que le nécessaire - ce qui réduit à la fois la taille des images et la **surface d'attaque** (moins de composants embarqués, moins de vulnérabilités potentielles).

Cette approche a été particulièrement précieuse dans un contexte de travail distant : elle m'a permis de disposer, depuis le Bénin, d'environnements strictement équivalents à ceux visés pour la production en France, sans dépendre d'une configuration manuelle propre à chaque machine.

5.3 Configuration et gestion des secrets (approche 12 facteurs)

J'ai appliqué le principe de la méthodologie **Twelve-Factor App** consistant à **externaliser toute la configuration dans l'environnement**, jamais dans le code. Un fichier .env.example documente l'ensemble des paramètres attendus, sans jamais contenir de valeurs sensibles réelles. La configuration couvre notamment :

- les **secrets JWT**, avec une distinction importante entre le secret de signature des **access tokens** et celui des **refresh tokens** : deux clés séparées, de sorte que la compromission de l'une n'affaiblit pas l'autre ;
- les paramètres des **cookies** (domaine, noms, durées de vie), du **CORS**, de la **limitation de débit** et de la **compression** ;
- l'accès à la base de données et à un cache **Redis** ;
- l'accès à la documentation d'API (**Swagger**), qui peut être **protégée par identifiants** afin de ne pas exposer publiquement la cartographie des endpoints.

Cette séparation stricte entre code et configuration est à la fois une bonne pratique Cloud (une même image se déploie sur plusieurs environnements) et une mesure de sécurité (les secrets ne transitent pas par le dépôt de code).

5.4 Déploiement et exploitation

La plateforme est en production et accessible publiquement à l'adresse <https://getcarpulse.com/>. Le frontend, bâti sur Next.js, se prête à un hébergement optimisé pour ce framework (build automatisé, diffusion via un réseau de distribution de contenu), tandis que l'API et les services de données sont conçus pour un déploiement conteneurisé.

L'exploitation s'appuie par ailleurs sur des points de **supervision** : le backend expose des points de contrôle de disponibilité (module health), et la brique de collecte dispose d'un tableau de bord de monitoring en temps réel. Ces éléments constituent le socle d'une démarche d'observabilité.

5.5 Sécurité de l'infrastructure et accès distant

La dimension Cloud rejoint ici la dimension cybersécurité, notamment du fait du travail à distance :

- **Isolation réseau et cloisonnement** : la séparation des services et des bases de données limite les mouvements latéraux en cas d'intrusion.
- **Accès distant maîtrisé** : opérer la plateforme depuis un autre pays impose une discipline sur les canaux d'accès (connexions chiffrées, moindre privilège, absence de secrets en clair).

- **Réduction de la surface d'attaque** : images minimales, exposition publique restreinte à la vitrine et à l'API strictement nécessaire, documentation d'API protégeable.

5.6 Maturité DevOps et perspectives d'industrialisation

Par souci d'honnêteté, je distingue ce qui est en place de ce qui constitue la feuille de route d'industrialisation que j'ai identifiée :

En place : conteneurisation des services, configuration et secrets externalisés, séparation stricte des environnements, points de supervision, mise en production effective.

Perspectives que je recommande :

- une **chaîne d'intégration et de déploiement continu (CI/CD)** exécutant automatiquement les tests, l'analyse statique et l'analyse de sécurité des dépendances (*SCA*) à chaque évolution ;
- une gestion des secrets par un **coffre-fort dédié** (*secrets manager*) plutôt que par de simples fichiers d'environnement ;
- une description de l'infrastructure sous forme de code (**IaC**) pour garantir la reproductibilité de l'hébergement ;
- une **orchestration** (par exemple via un orchestrateur de conteneurs) pour la mise à l'échelle, et une **observabilité** consolidée (journaux, métriques, alertes).

Ces perspectives montrent que le projet est arrivé à un stade fonctionnel et sécurisé, et qu'il dispose d'une trajectoire d'industrialisation claire - trajectoire que ma double compétence cybersécurité / Cloud me permet de porter.

VI - Bilan de l'alternance

6.1 Compétences acquises et développées

Cette alternance m'a permis de développer des compétences sur quatre registres complémentaires, en lien avec le référentiel de mon titre.

Pilotage de projet et product management : En tant que chef de projet, analyste fonctionnel et product manager, j'ai piloté un projet complet de bout en bout : recueil et formalisation du besoin, définition des parcours utilisateurs, priorisation d'une feuille de route, coordination d'une équipe pluridisciplinaire (data, backend, frontend) et animation des points d'avancement. J'ai appris à arbitrer entre ambition fonctionnelle, exigence de sécurité et contraintes de délai.

Cybersécurité : J'ai conçu et implémenté une chaîne d'authentification robuste (bcrypt, JWT à double jeton, rotation avec détection de réutilisation, gestion des sessions et des appareils), un contrôle d'accès multi-niveaux (RBAC, protection contre l'IDOR), et une défense en profondeur applicative (validation, en-têtes de sécurité, limitation de débit). J'ai également mené une réflexion sur les enjeux juridiques et RGPD de la collecte de données, et formalisé une analyse de risques inspirée de l'OWASP.

Cloud et DevOps : J'ai conteneurisé les services, externalisé la configuration et les secrets selon la méthodologie *12 facteurs*, et défini une architecture orientée services propice à un déploiement Cloud maîtrisé, jusqu'à la mise en production effective de la plateforme.

Ingénierie logicielle : J'ai mis en œuvre une architecture *Domain-Driven Design* rigoureuse, des tests automatisés et des outils de qualité de code, dans un écosystème technique moderne (NestJS, Prisma, Next.js, Python).

À ces compétences techniques s'ajoute une compétence transversale déterminante : la capacité à **collaborer efficacement à distance**, à l'international, en autonomie et avec rigueur.

6.2 Difficultés rencontrées et solutions apportées

Concilier vitesse de livraison et exigence de sécurité : La principale tension d'un projet produit est d'avancer vite sans sacrifier la sécurité. J'ai résolu cette tension par une approche *security by design* : en intégrant les exigences de sécurité dès la conception des modules, elles ne sont pas devenues une dette à rembourser plus tard.

Fiabiliser l'évaluation des annonces : Aucune méthode d'analyse n'est infaillible. Le choix d'un consensus à trois méthodes indépendantes a permis de

réduire le risque d'erreur systématique et d'apporter une explicabilité au verdict, plutôt que de dépendre d'une « boîte noire ».

Travailler à distance et de façon asynchrone : Le décalage et la distance ont exigé une communication écrite claire, une documentation soignée et des environnements reproductibles. La conteneurisation et l'externalisation de la configuration ont directement répondu à ce défi.

Robustesse de la collecte : Les sites cibles évoluent et se protègent. La double stratégie d'extraction (données structurées puis repli HTML) et les phases de mise à jour/archivage ont fiabilisé la collecte dans la durée.

6.3 Apports pour l'entreprise et perspectives

Pour Bioénergie, CarPulse constitue un **outil d'aide à la décision** directement utile à l'activité d'achat-revente : il automatise et objective le repérage des véhicules sous-évalués, une tâche jusque-là manuelle et chronophage. La plateforme est fonctionnelle et en production.

Les perspectives d'évolution que j'ai identifiées sont de trois ordres :

- **Fonctionnelles** : alertes personnalisées, extension à d'autres places de marché, affinage des modèles d'analyse.
- **Sécurité** : authentification multifacteur, journalisation de sécurité centralisée, tests de sécurité automatisés.
- **Industrialisation** : chaîne CI/CD complète, gestion des secrets par coffre-fort, infrastructure décrite en code et observabilité consolidée.

Ces axes prolongent naturellement le travail réalisé et s'inscrivent dans la continuité de mes deux spécialités.

Conclusion

Le projet CarPulse aura été, tout au long de cette alternance chez Bioénergie, un terrain d'application idéal pour mes deux spécialités. Parti d'un besoin métier concret - repérer automatiquement les bonnes affaires sur le marché de l'occasion automobile - j'ai pu piloter et construire une plateforme complète, de la collecte de données jusqu'à une application web sécurisée en production.

La problématique posée en introduction était la suivante : *comment concevoir, sécuriser et industrialiser une plateforme d'intelligence de marché manipulant des données massives et des comptes utilisateurs, tout en garantissant sa conformité, sa maintenabilité et un déploiement maîtrisé ?* La réponse apportée par CarPulse tient en trois principes qui se sont révélés être les fils conducteurs de mon travail :

1. **La sécurité par conception** - une authentification robuste, un contrôle d'accès multi-niveaux et une défense en profondeur pensés dès l'architecture, et non ajoutés après coup ;
2. **La redondance maîtrisée** - qu'il s'agisse du consensus à trois méthodes d'analyse ou de la superposition de couches de sécurité indépendantes, la fiabilité naît de la diversité et de la redondance ;
3. **L'industrialisation reproductible** - une architecture modulaire et conteneurisée, une configuration externalisée, qui rendent le déploiement maîtrisé et la trajectoire d'évolution claire.

Sur le plan personnel, cette expérience m'a permis d'assumer un rôle transverse exigeant - à la fois chef de projet, référent sécurité et référent Cloud - et de le faire dans un contexte de télétravail international qui a renforcé mon autonomie et ma rigueur. Elle confirme mon projet professionnel : exercer à l'intersection de la cybersécurité et du Cloud, là où la sécurité se conçoit et s'opère à l'échelle de systèmes entiers.

Les perspectives d'évolution identifiées - authentification multifacteur, journalisation de sécurité, chaîne CI/CD et observabilité - dessinent la suite naturelle de ce travail, que je serais heureux de présenter et d'approfondir lors de la soutenance.

Glossaire et acronymes

Terme	Définition
API	<i>Application Programming Interface</i> - interface de programmation exposant des services.

Terme	Définition
JWT	<i>JSON Web Token</i> - jeton signé transportant des informations d'authentification.
Access/Refresh token	Jeton d'accès de courte durée / jeton de renouvellement de longue durée.
Bcrypt	Algorithme de hachage de mots de passe conçu pour être lent (résistant au <i>brute-force</i>).
RBAC	<i>Role-Based Access Control</i> - contrôle d'accès fondé sur les rôles.
DDD	<i>Domain-Driven Design</i> - conception logicielle pilotée par le domaine métier.
ORM	<i>Object-Relational Mapping</i> - correspondance objet-relationnel (ici Prisma).
RGPD	Règlement Général sur la Protection des Données.
OWASP	<i>Open Worldwide Application Security Project</i> - référentiel de sécurité applicative.
CI/CD	<i>Continuous Integration / Continuous Deployment</i> - intégration et déploiement continus.
IaC	<i>Infrastructure as Code</i> - description de l'infrastructure sous forme de code.
Conteneur / Docker	Unité logicielle isolée embarquant application et dépendances.
Scraping	Collecte automatisée de données depuis des pages web.

Terme	Définition
UC Mode	<i>Undetected Chrome Mode</i> - mode furtif de navigateur automatisé.
ML	<i>Machine Learning</i> - apprentissage automatique.
httpOnly	Attribut de cookie interdisant son accès depuis JavaScript (anti-XSS).

Bibliographie

- OWASP Foundation, *OWASP Top 10* et *OWASP API Security Top 10*.
- OWASP Foundation, *Cheat Sheet Series* - notamment *Authentication*, *JSON Web Token for Java*, *Password Storage*, *Session Management*.
- IETF, *RFC 7519 - JSON Web Token (JWT)* et *RFC 6749 - The OAuth 2.0 Authorization Framework*.
- CNIL, recommandations relatives au RGPD et à la collecte de données par moissonnage (*web scraping*).
- A. Wiggins, *The Twelve-Factor App*.
- Documentation officielle : NestJS, Prisma, Next.js, Docker.

Annexes

- **Annexe A** - Schéma d'architecture global de la plateforme (les briques et leurs flux).
- **Annexe B** - Diagramme du flux d'authentification (connexion, émission des jetons, rotation et détection de réutilisation).
- **Annexe C** - Modèle de données simplifié (utilisateurs, sessions, appareils, jetons).
- **Annexe D** - Démo de l'application (espaces USER et ADMIN) et de la vitrine getcarpulse.com.

- **Annexe E** - Extrait de code commenté illustrant la rotation des refresh tokens avec détection de réutilisation.
- **Annexe F** - Tableau récapitulatif de l'analyse de risques (menaces / mesures / améliorations).